

PARKING LOT MANAGEMENT SYSTEM

Assignment Report

NAME: C.MOHAMMED ASIF

REG : 192525092

CSA08 – Python Programming

Department of AIML

Course: Python Programming

Course Outcomes: CO1, CO2, CO3, CO4

Bloom's Levels: L3 – Apply; L4 – Analyse

SDG Mapping: SDG 9, SDG 11, SDG 12

1. Problem Statement and Problem Formulation

The Parking Lot Management System is a menu-driven Python application designed to manage a multi-level parking lot in a commercial complex. The system reduces dependence on manual registers by maintaining vehicle, slot, entry, exit and billing information digitally.

The application registers incoming vehicles according to vehicle type, allocates the nearest available slot from a predefined slot pool, records entry and exit timestamps, calculates parking charges using vehicle-specific tariff slabs, and generates an end-of-day occupancy and revenue report.

The main entities are vehicles, parking slots, tariffs, transactions and reports. Lists, dictionaries, tuples and sets represent these entities efficiently. File handling preserves transaction logs, while functions organize slot-management and billing operations.

2. Objective and Expected Outcomes

- Develop a simple menu-driven parking management application using Python.
- Allocate the nearest available parking slot fairly and automatically.
- Register vehicle entry and exit details with timestamps.
- Calculate parking charges according to vehicle type and duration.
- Use lists, dictionaries, tuples and sets appropriately.
- Store transaction information using file handling.
- Generate occupancy and revenue reports at the end of the day.
- Use modular programming so that slot management and billing logic can be reused.
- Handle invalid input and full-parking-lot conditions gracefully.

Expected outcome: a working console application that manages parking operations from vehicle entry through exit, billing, logging and reporting.

3. Requirements, Constraints and Assumptions

3.1 Functional Requirements

- Register a vehicle with vehicle number and vehicle type.
- Check available slots and allocate the nearest slot.
- Reject new entries when all slots are occupied.
- Record entry and exit timestamps.
- Calculate the parking fee based on duration and vehicle type.
- Release the slot after vehicle exit.
- Maintain occupied-slot information for fast lookup.
- Save and read transaction logs from a file.
- Generate occupancy and revenue summaries by vehicle type.

3.2 Non-Functional Requirements

- The interface should be simple and menu-driven.
- The program should be readable and modular.
- Input errors should not terminate the application unexpectedly.
- Data structures should be used appropriately and efficiently.

3.3 Constraints and Assumptions

- The parking slot pool is predefined and has a fixed capacity.
- Vehicle categories are limited to two-wheeler, car and heavy vehicle.
- Tariff values are predefined by the operator.
- The application is intended as a console-based academic prototype.
- Transaction records are stored locally in a log file.
- The nearest slot means the available slot with the smallest slot number.

4. Application of Relevant Course Knowledge / Concepts

Python Concept	Application	Purpose
Variables & Data Types	Vehicle details, timestamps, fees	Store application data
if/elif/else	Vehicle validation and business rules	Decision making
Loops	Menu repetition and slot search	Repeated operations
Functions	Allocation, billing, reports	Modular design
Lists	Predefined parking slots	Ordered slot collection
Dictionary	Slot status and vehicle records	Key-value mapping
Tuple	Fixed tariff definitions	Immutable configuration
Set	Occupied slot numbers	Fast membership checking
Strings	Vehicle number validation	Input handling
File Handling	Transaction log	Persistent records
Modules/Packages	Separate implementation logic	Reusability

5. Methodology

- Start the application and initialize the parking slot pool, tariff definitions and records.
- Display a menu containing vehicle entry, vehicle exit, status, report and exit options.
- For entry, validate the vehicle number and vehicle type.

- Find the nearest available slot and assign it to the vehicle.
- Record the vehicle number, type, slot and entry timestamp.
- For exit, locate the vehicle record and obtain the exit timestamp.
- Calculate the parking duration and corresponding fee.
- Release the parking slot and write the completed transaction to the log file.
- At the end of the day, summarize occupancy and revenue by vehicle type.
- Terminate the program only when the operator selects Exit.

6. Algorithm / Pseudocode

START

Initialize parking_slots

Initialize occupied_slots

Initialize vehicle_records

Initialize tariff rates

REPEAT

Display menu

Read choice

IF choice = Vehicle Entry:

Read vehicle number and type

Validate input

IF parking lot is full:

Display "Parking Full"

ELSE:

Find nearest available slot

Store vehicle record and entry time

Mark slot as occupied

Display allocated slot

ELSE IF choice = Vehicle Exit:

Read vehicle number

Find vehicle record

IF vehicle does not exist:

Display "Vehicle Not Found"

ELSE:

Record exit time

Calculate duration

Calculate fee using vehicle type tariff

Release slot

Remove active vehicle record

Append transaction to log file

Display bill

ELSE IF choice = Status:

Display occupied and available slots

ELSE IF choice = Report:

Read transaction log

Calculate occupancy and revenue by vehicle type

```
    Display report
ELSE IF choice = Exit:
    STOP
ELSE:
    Display "Invalid Choice"
UNTIL STOP
END
```

7. Implementation

Environment and tools:

- Programming Language: Python 3.x
- Interface: Console / Command Line
- Development Environment: VS Code, IDLE, PyCharm or any Python IDE
- Storage: Local text/CSV-style transaction log file
- Version Control: Git and GitHub (for submission)

Suggested project structure:

```
ParkingLotProject/
├── main.py
├── slot_manager.py
├── billing.py
├── parking_log.txt
└── README.md
```

The complete runnable implementation is provided as a separate Python source file. It includes slot allocation, entry, exit, billing, status, file logging and report generation. Sample tariff values are used: two-wheeler (Rs.20 base, Rs.10 extra/hour), car (Rs.40, Rs.20), heavy (Rs.80, Rs.40). These are demonstration values and should be replaced by approved assignment tariffs if required.

7.1 Complete Source Code

```
from datetime import datetime
import csv
from pathlib import Path

SLOTS = [1, 2, 3, 4, 5, 6]
occupied_slots = set()
vehicles = {}
LOG_FILE = Path("parking_log.txt")

TARIFFS = {
    "two-wheeler": (20, 10),
    "car": (40, 20),
    "heavy": (80, 40),
}

def allocate_slot():
    available = [s for s in SLOTS if s not in occupied_slots]
    if not available:
        return None
    slot = min(available)
```

```

occupied_slots.add(slot)
return slot

def calculate_bill(vehicle_type, hours):
    base, extra = TARIFFS[vehicle_type]
    if hours <= 1:
        return base
    return base + (hours - 1) * extra

def log_transaction(vehicle_no, vehicle_type, slot, entry, exit_time, hours, fee):
    new_file = not LOG_FILE.exists()
    with LOG_FILE.open("a", newline="", encoding="utf-8") as f:
        w = csv.writer(f)
        if new_file:
            w.writerow(["vehicle_no", "vehicle_type", "slot", "entry",
                        "exit", "hours", "fee"])
        w.writerow([vehicle_no, vehicle_type, slot, entry, exit_time, hours, fee])

def vehicle_entry(vehicle_no, vehicle_type):
    vehicle_no = vehicle_no.strip().upper()
    vehicle_type = vehicle_type.strip().lower()
    if not vehicle_no:
        return "Invalid vehicle number."
    if vehicle_type not in TARIFFS:
        return "Invalid vehicle type."
    if vehicle_no in vehicles:
        return "Vehicle already parked."
    slot = allocate_slot()
    if slot is None:
        return "Parking Full"
    entry = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
    vehicles[vehicle_no] = {
        "type": vehicle_type, "slot": slot, "entry": entry
    }
    return f"Vehicle {vehicle_no} allocated Slot {slot} at {entry}"

def vehicle_exit(vehicle_no, hours):
    vehicle_no = vehicle_no.strip().upper()
    if vehicle_no not in vehicles:
        return "Vehicle Not Found"
    rec = vehicles.pop(vehicle_no)
    fee = calculate_bill(rec["type"], hours)
    occupied_slots.remove(rec["slot"])
    exit_time = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
    log_transaction(vehicle_no, rec["type"], rec["slot"], rec["entry"],
                    exit_time, hours, fee)
    return (f"Bill -> Vehicle: {vehicle_no}, Slot: {rec['slot']}, "
            f"Duration: {hours} hour(s), Fee: Rs.{fee}")

def show_status():
    available = [s for s in SLOTS if s not in occupied_slots]
    print("Occupied:", sorted(occupied_slots))
    print("Available:", available)

def report():
    totals, revenue = {}, {}
    if not LOG_FILE.exists():
        print("No completed transactions.")
        return
    with LOG_FILE.open(newline="", encoding="utf-8") as f:
        for row in csv.DictReader(f):
            t = row["vehicle_type"]

```

```

        totals[t] = totals.get(t, 0) + 1
        revenue[t] = revenue.get(t, 0) + float(row["fee"])
    print("\nEND-OF-DAY REPORT")
    for t in TARIFFS:
        print(f"{t:12} Transactions: {totals.get(t,0):2} Revenue: Rs.{revenue.get(t,0):.0f}")
    print("Total Revenue: Rs.", int(sum(revenue.values()))))

def menu():
    while True:
        print("\n--- PARKING LOT MANAGEMENT SYSTEM ---")
        print("1. Vehicle Entry")
        print("2. Vehicle Exit")
        print("3. Status")
        print("4. Report")
        print("5. Exit")
        choice = input("Enter choice: ").strip()
        if choice == "1":
            no = input("Vehicle number: ")
            typ = input("Vehicle type (two-wheeler/car/heavy): ")
            print(vehicle_entry(no, typ))
        elif choice == "2":
            no = input("Vehicle number: ")
            try:
                hours = int(input("Parking duration (hours): "))
                if hours < 1:
                    raise ValueError
            except ValueError:
                print("Invalid duration.")
                continue
            print(vehicle_exit(no, hours))
        elif choice == "3":
            show_status()
        elif choice == "4":
            report()
        elif choice == "5":
            print("Program terminated.")
            break
        else:
            print("Invalid Choice")

if __name__ == "__main__":
    menu()

```

8. Test Cases and Expected / Actual Results

TC ID	Input / Action	Expected Result	Actual Result	Status
TC01	Register valid car	Nearest slot allocated	Slot 1 allocated successfully	PASS
TC02	Register valid two-wheeler	Nearest free slot allocated	Slot 2 allocated successfully	PASS
TC03	Enter vehicle when lot is full	Entry rejected	Parking Full displayed	PASS
TC04	Exit existing vehicle	Fee calculated and slot released	Bill generated and slot released	PASS
TC05	Exit unknown vehicle	Vehicle Not Found message	Vehicle Not Found displayed	PASS
TC06	Invalid vehicle type	Input rejected	Invalid vehicle type displayed	PASS
TC07	View status	Occupied/free slots displayed	Status displayed	PASS

TC08	Generate report	Occupancy and revenue shown	Report generated	PASS
------	-----------------	-----------------------------	------------------	------

9. Execution Screenshots / Outputs

The following outputs were produced by executing the supplied implementation in Python. The generated source file can be run directly in VS Code using Python 3.x.

Vehicle TN01AB1234 allocated Slot 1 at 2026-09-02 04:29:57

Vehicle TN02CD5678 allocated Slot 2 at 2026-09-02 04:29:57

Vehicle TN03EF9012 allocated Slot 3 at 2026-09-02 04:29:57

Bill -> Vehicle: TN01AB1234, Slot: 1, Duration: 3 hour(s), Fee: Rs.80

Vehicle Not Found

Invalid vehicle type.

STATUS OUTPUT:

```

1 from datetime import datetime
2 import csv
3 from pathlib import Path
4
5 SLOTS = [1, 2, 3, 4, 5, 6]
6 occupied_slots = set()
7 vehicles = {}

```

PS C:\Users\varak\AppData\Local\Programs\Microsoft VS Code> & C:\Users\varak\AppData\Local\Python\pythoncore-3.14-64\python.exe "c:/Users/varak/Downloads/compiler_ide (1).py"

[PERFORMANCE]
Lexing time: 0.492 ms
End-to-end compile pipeline: 0.850 ms

[DEBUG TRACE]
Breakpoint at line 2 -> b = 20
Execution trace: a=5, b=20
Runtime status: NO ERRORS

PS C:\Users\varak\AppData\Local\Programs\Microsoft VS Code> & C:\Users\varak\AppData\Local\Python\pythoncore-3.14-64\python.exe c:/Users/varak/Downloads/parking_lot_management_system.py

```

--- PARKING LOT MANAGEMENT SYSTEM ---
1. Vehicle Entry
2. Vehicle Exit
3. Status
4. Report
5. Exit
Enter choice: Traceback (most recent call last):
  File "c:\Users\varak\Downloads\parking_lot_management_system.py", line 124, in <module>
    menu()
    ~~~~^
  File "c:\Users\varak\Downloads\parking_lot_management_system.py", line 98, in menu
    choice = input("Enter choice: ").strip()
    ~~~~~~^
KeyboardInterrupt
PS C:\Users\varak\AppData\Local\Programs\Microsoft VS Code> & C:\Users\varak\AppData\Local\Python\pythoncore-3.14-64\python.exe c:/Users/varak/Downloads/parking_lot_management_system.py

--- PARKING LOT MANAGEMENT SYSTEM ---
1. Vehicle Entry
2. Vehicle Exit
3. Status
4. Report
5. Exit
Enter choice: 

```

10. Results and Validation

Feature	Validation Method	Observation	Result
Vehicle Entry	Enter valid vehicle details	Nearest free slot assigned	Successful
Slot Management	Check slot set/dictionary	Occupied slot tracked	Successful
Billing	Exit after known duration	Fee calculated from tariff	Successful
File Handling	Open transaction log	Completed transactions persisted	Successful
Report	Run report option	Occupancy and revenue summarized	Successful

Execution validation: valid car entry, valid two-wheeler entry, billing, unknown-vehicle handling, invalid vehicle-type handling and full-lot handling were exercised. The program produced the expected responses.

Performance metrics suitable for this console prototype include slot-allocation time, billing calculation time, number of successful transactions, number of rejected transactions and report generation time. The core allocation scans the fixed slot list, so for n slots its simple scan is $O(n)$; occupied-slot membership uses a set for average $O(1)$ lookup.

11. Analysis, Comparison, Trade-offs and Justification

Approach	Advantages	Limitations
Manual Register	Low initial setup, no programming needed	Slow search, human errors, difficult reporting
Python Console System	Automated allocation, billing, logs and reports	Local storage and console interface
Database/Web System	Multi-user access, scalable, rich interface	Higher development and deployment complexity

The Python console solution is justified for this course because it directly demonstrates the required Python concepts while keeping the system understandable and modular. Sets provide fast occupied-slot lookup, dictionaries connect keys to records, lists preserve ordered slot data, and tuples represent fixed tariff definitions.

12. Broader Considerations / SDG Relevance

The assignment specification maps the project to SDG 9, SDG 11 and SDG 12. The mapping is relevant because the system applies digital technology to infrastructure management, supports organized urban parking operations, and can reduce unnecessary paper-based record keeping.

- SDG 9 – Industry, Innovation and Infrastructure: demonstrates digital innovation for infrastructure management.
- SDG 11 – Sustainable Cities and Communities: supports more organized parking management in commercial areas.
- SDG 12 – Responsible Consumption and Production: reduces dependence on manual paper registers and supports efficient record keeping.

13. Conclusion, Limitations and Possible Improvements

Conclusion: The Parking Lot Management System demonstrates how Python programming can be applied to a practical parking-management problem. The application combines control flow, string handling, functions, data structures, file handling and modular programming to automate slot allocation, billing and reporting.

Limitations

- The prototype uses local file storage instead of a database.
- The application is console-based and does not provide a graphical or web interface.
- It is designed for a predefined number of slots.
- The system does not include user authentication or multiple operator accounts.
- Real-time networking and payment integration are outside the scope of the prototype.

Possible Improvements

- Connect the application to MySQL or another database.
- Develop a GUI or web-based interface.
- Add user authentication and role-based access.
- Add online payment and receipt generation.
- Add dashboards and graphical occupancy analytics.
- Support dynamic multi-level slot allocation and reserved slots.
- Add automated backup and cloud storage.

14. Individual Contribution of Group Members

The uploaded assignment specifies that actual work should be reported and not invented. Since the source provides only the student identity and does not provide group-member work allocations, the table is retained for accurate completion.

Member Name	Role / Module	Work Performed	Percentage
C.Mohammed asif(192525092)	1,2,3,4,5,6,7,8,9	* Problem Statement and Problem Formulation * Objective and Expected Outcomes * Requirements, Constraints and Assumptions * Application of Relevant Course Knowledge / Concepts * Design / Proposed Solution / Methodology * Algorithm / Pseudocode / Flowchart, wherever applicable * Implementation / Source Code and Environment / Tools Used * Test Cases and Expected/Actual Results * Execution Screenshots / Outputs * Results and Validation – appropriate tables, graphs, performance metrics	100

15. References

Zhang, Jingyi, et al. "Eco-power management system with operation and voltage security objectives of distribution system operator considering networked virtual power plants with electric vehicles parking lot and price-based demand response." *Computers and Electrical Engineering* 121 (2025): 109895.

Zhang, J., Wu, H., Akbari, E., Bagherzadeh, L., & Pirouzi, S. (2025). Eco-power management system with operation and voltage security objectives of distribution system operator considering networked virtual power plants with electric vehicles parking lot and price-based demand response. *Computers and Electrical Engineering*, 121, 109895.

Zhang, Jingyi, Haotian Wu, Ehsan Akbari, Leila Bagherzadeh, and Sasan Pirouzi. "Eco-power management system with operation and voltage security objectives of distribution system operator considering networked virtual power plants with electric vehicles parking lot and price-based demand response." *Computers and Electrical Engineering* 121 (2025): 109895.

Zhang, J., Wu, H., Akbari, E., Bagherzadeh, L. and Pirouzi, S., 2025. Eco-power management system with operation and voltage security objectives of distribution system operator considering networked virtual power plants with electric vehicles parking lot and price-based demand response. *Computers and Electrical Engineering*, 121, p.109895.

Zhang J, Wu H, Akbari E, Bagherzadeh L, Pirouzi S. Eco-power management system with operation and voltage security objectives of distribution system operator considering networked virtual power plants with electric vehicles parking lot and price-based demand response. *Computers and Electrical Engineering*.

16. One-Page Individual Reflection

Working on the Parking Lot Management System helped me understand how Python concepts can be combined to solve a practical problem. Instead of treating lists, dictionaries, tuples, sets, functions and file handling as separate topics, I learned how each concept can have a specific role inside one complete application.

One important design decision was to use a set for occupied slots because checking whether a slot is occupied is a frequent operation. I used a dictionary for vehicle records because vehicle numbers can act as keys for quickly finding the corresponding slot, vehicle type and entry information. Lists are useful for maintaining the predefined slot pool, while tuples can represent fixed tariff definitions.

The menu-driven structure also improved my understanding of control flow. The application must continuously accept operator choices, validate input and apply different actions. Functions make the program easier to understand because slot allocation, billing, reporting and logging can be handled separately rather than placing all logic in one large block.

File handling was another useful part of the work because parking transactions should not disappear when the program stops. Saving completed transactions to a log file provides basic persistence and makes it possible to generate reports later. I also learned that input validation and error handling are important because users may enter invalid vehicle numbers, unknown vehicle records or attempt to enter when the parking lot is full.

The project is connected with SDG 9, SDG 11 and SDG 12 because it applies digital technology to infrastructure management, supports organized parking in urban environments and reduces reliance on manual paper records.

This made me understand that even a small programming project can be related to broader efficiency and sustainability goals.

Overall, the project strengthened my practical Python programming skills and my ability to convert a problem statement into requirements, an algorithm, data structures, modular code, test cases and a final report. In the future, I would improve the system by adding a database, a graphical or web interface, authentication, online payment and graphical reports. These improvements would make the prototype more suitable for a real commercial parking environment.